

作業ログ

報告者：成毛海人

報告日：2026/05/27

プロジェクト名：受動的環境介入によるメンタルヘルス支援：大学空間における自動演奏装置の提案

1. 進捗サマリー（要約）

- 全体進捗率：40%
- マイルストーン達成状況：前倒し
- 主要成果：
 - － 成果1：オルガンのサンプル音源の周波数解析から、音色作成で実装すべき音圧設定（倍音ごと）を決定。
 - － 成果2：パーカッション楽器（キックドラム・ハイハット）のサンプル音源の再生経過時間の区間（ヒット直後・減衰中盤・減衰後）ごとの周波数解析から、音の減衰のタイミングと周波数変化を確認。
- 重大課題（影響度：高）
 - － 課題内容：オルガンのサンプル音源と比べて、作成した音は厚みが足りずこもっている。

2. 作業ログ（詳細トラッキング）

作業項目	開始日	予定完了日	状態	進捗率	依存関係・ブロック	コメント
オルガン倍音構造の調査	4/26	5/6	完了	100%	なし	なし
パーカッション楽器の発音原理の調査	4/30	5/13	完了	100%	なし	なし
オルガン音色作成	5/4	6/24	進行中	50%	マスター・スレーブ・Processing 間通信の完成待ち	通信担当者の作業完了まで音色改良を進める

3. 課題管理

課題	発生原因	影響度	優先度	対策	期限	状態
オルガン再現音の厚みの不足	生成するサイン波の数が足りない	高	高	サイン波を増やす	6/3	対応中

4. AI 利用状況と検証

4.1 利用内容

- 利用目的：コード修正提案
- 入力（プロンプト要旨）：
 - － 修正前のプログラムコード（あまり近似しなかった音）を記述し，なぜ音の厚みが足りないかを質問した。
- 出力概要：
 - － 倍音構造が単純であるため，サイン波をさらに追加して音の密度を上げることを提案された。

4.2 検証方法

- 検証手法：修正前と修正後の音声の FFT スペクトルを，サンプル音源の FFT スペクトルとそれぞれ比較
- 参照資料：倍音構造について [1]，加算合成とオルガンの関係 [2]
- 検証手順：
 - － 1：サンプル音源を Audacity で開き FFT スペクトル解析
 - － 2：修正前の作成音声を同様に FFT スペクトル解析
 - － 3：修正後の作成音声を同様に FFT スペクトル解析
 - － 4：スペクトル表示で得た各波形を比較し，サンプル波形と修正後の波形が修正前より近似しているかを確認

4.3 ファクトチェック結果

- 一致点：波形スペクトルがサンプル音源のものにより近づき，音圧の増加を確認できた。
- 不一致・疑義：音圧は上がったものの，音の全体的なこもりはまだ拭えられなかった。
- 最終判断：採用
- 根拠：FFT スペクトル表示の結果から，音圧増加には効果的な手法だと捉えたため。

5. 次回計画

- 目標：作成音の音質の改善（サンプル音源同様の音圧や音色に近づける）
- 達成基準：最終的に完成した音声を FFT 解析し，サンプル音源の波形と概ね一致すること。
- 期限：6/24

6. その他

通信方式作成の担当者が通信プログラムを完成するまでには，オルガン音声だけでなく，パーカッション音声もなるべく再現度を高めた上で完成まで持っていく。

7. 付録

7.1 添付資料：オルガン音生成プログラム

Listing 1 オルガン音生成プログラム（修正前）

```
1 import ddf.minim.*;
2 import ddf.minim.ugens.*;
3
```

```

4 Minim minim;
5 AudioOutput out;
6 Waveform currentWaveform;
7
8 String[] melody = {
9     "C2"
10 };
11
12 float[] duration = {
13     40f
14 };
15
16 float[] startTime = {
17     0.0f
18 };
19
20 class HackInstrument implements Instrument
21 {
22     Summer mix;
23     ADSR adsr;
24
25     HackInstrument(float frequency, float maxAmp, Waveform wf)
26     {
27         mix = new Summer();
28
29         float detune = 3.0f; // cent
30
31         float f1 = frequency * pow(2, -detune / 1200.0f);
32         float f2 = frequency;
33         float f3 = frequency * pow(2, detune / 1200.0f);
34
35         Oscil osc1 = new Oscil(f1, maxAmp / 3.0f, wf);
36         Oscil osc2 = new Oscil(f2, maxAmp / 3.0f, wf);
37         Oscil osc3 = new Oscil(f3, maxAmp / 3.0f, wf);
38
39         osc1.patch(mix);
40         osc2.patch(mix);
41         osc3.patch(mix);
42
43         adsr = new ADSR(maxAmp, 0.08f, 0.0f, 1.0f, 0.3f); mix.patch(adsr);
44     }
45
46     void noteOn(float duration)
47     {
48         adsr.patch(out);
49         adsr.noteOn();
50     }
51
52     void noteOff()
53     {
54         adsr.noteOff();
55     }
56 }
57

```

```

58 void setup()
59 {
60     size(512, 200);
61     minim = new Minim(this);
62     out = minim.getLineOut();
63     out.setTempo(120);
64     currentWaveform = Waves.SINE;
65 }
66
67 void playSong()
68 {
69     out.pauseNotes();
70     for (int i = 0; i < melody.length; i++)
71     {
72         out.playNote(
73             startTime[i],
74             duration[i],
75             new HackInstrument(
76                 Frequency.ofPitch(melody[i]).asHz(),
77                 0.5f,
78                 currentWaveform
79             )
80         );
81     }
82     out.resumeNotes();
83 }
84
85 void draw()
86 {
87     background(0);
88     stroke(255);
89
90     for (int i = 0; i < out.bufferSize() - 1; i++)
91     {
92         line(i, 50 - out.left.get(i) * 50,
93             i + 1, 50 - out.left.get(i + 1) * 50);
94
95         line(i, 150 - out.right.get(i) * 50,
96             i + 1, 150 - out.right.get(i + 1) * 50);
97     }
98 }
99
100 void keyPressed()
101 {
102     switch(key)
103     {
104     case '1':
105         currentWaveform = WavetableGenerator.gen10(
106             8192,
107             new float[] {0.5f, 0.8f, 0.5f, 0.8f, 0.5f, 0.65f, 0.5f, 0.9f}
108         );
109         break;
110     case 'p':
111         playSong();

```

```

112     break;
113 }
114 }

```

Listing 2 オルガン音生成プログラム（修正後）

```

1  import ddf.minim.*;
2  import ddf.minim.ugens.*;
3
4  Minim minim;
5  AudioOutput out;
6  Waveform currentWaveform;
7
8  String[] melody = {
9      "C2"
10 };
11
12 float[] duration = {
13     20f
14 };
15
16 float[] startTime = {
17     0.0f
18 };
19
20 class HackInstrument implements Instrument
21 {
22     Summer mix;
23     ADSR adsr;
24
25     HackInstrument(float frequency, float maxAmp, Waveform wf)
26     {
27         mix = new Summer();
28
29         float f1 = frequency * pow(2, -6.0f / 1200.0f);
30         float f2 = frequency * pow(2, -2.0f / 1200.0f);
31         float f3 = frequency * pow(2, 2.0f / 1200.0f);
32         float f4 = frequency * pow(2, 6.0f / 1200.0f);デチューン
33
34         //(関数内 pow)は後に調整予定
35
36         Oscil osc1 = new Oscil(f1, maxAmp/4.0f, wf);
37         Oscil osc2 = new Oscil(f2, maxAmp/4.0f, wf);
38         Oscil osc3 = new Oscil(f3, maxAmp/4.0f, wf);
39         Oscil osc4 = new Oscil(f4, maxAmp/4.0f, wf);
40
41         osc1.patch(mix);
42         osc2.patch(mix);
43         osc3.patch(mix);
44         osc4.patch(mix);
45
46         adsr = new ADSR(maxAmp, 0.08f, 0.0f, 1.0f, 0.3f);
47
48         mix.patch(adsr);

```

```

49     }
50
51     void noteOn(float duration)
52     {
53         adsr.patch(out);
54         adsr.noteOn();
55     }
56
57     void noteOff()
58     {
59         adsr.noteOff();
60     }
61 }
62
63 void setup()
64 {
65     size(512, 200);
66     minim = new Minim(this);
67     out = minim.getLineOut();
68     out.setTempo(120);
69     currentWaveform = Waves.SINE;
70 }
71
72 void playSong()
73 {
74     out.pauseNotes();
75     for (int i = 0; i < melody.length; i++)
76     {
77         out.playNote(
78             startTime[i],
79             duration[i],
80             new HackInstrument(
81                 Frequency.ofPitch(melody[i]).asHz(),
82                 0.5f,
83                 currentWaveform
84             )
85         );
86     }
87     out.resumeNotes();
88 }
89
90 void draw()
91 {
92     background(0);
93     stroke(255);
94
95     for (int i = 0; i < out.bufferSize() - 1; i++)
96     {
97         line(i, 50 - out.left.get(i) * 50,
98             i + 1, 50 - out.left.get(i + 1) * 50);
99
100         line(i, 150 - out.right.get(i) * 50,
101             i + 1, 150 - out.right.get(i + 1) * 50);
102     }

```

```
103 }  
104  
105 void keyPressed()  
106 {  
107     switch(key)  
108     {  
109         case '1':  
110             currentWaveform = WavetableGenerator.gen10(  
111                 8192,  
112                 new float[] {0.5f, 0.8f, 0.5f, 0.8f, 0.5f, 0.65f, 0.5f, 0.9f}  
113             );  
114             break;  
115         case 'p':  
116             playSong();  
117             break;  
118     }  
119 }
```

参考文献

- [1] ライブ UtaTen, 「倍音とは一体？倍音を構造する 3 要素や効果を超簡単に解説！ 2026 年 5 月」,
閲覧日：2026 年 5 月 25 日
<https://utaten.com/live/overtones/>
- [2] SOUND HOUSE, 「蠱惑の楽器たち 70. 電子音源の仕組み 10 加算合成」,
閲覧日：2026 年 5 月 25 日
https://www.soundhouse.co.jp/contents/column/index?post=3419&srsltid=AfmB0orKApb-9hmJRay7__BTHPV0ZZu9td-Sv5cgLcaHrWyM401_zRYp